
Programspråkssemantik
Laborationsmaterial 98/99



Introduktion till Gofer-Implementationerna

Vi börjar med några grundläggande definitioner för att kunna implementera de olika semantikstilarna i Gofer. Definitionerna baserar sig på materialet i kapitel 1 och finns på hemsidan under namnet `appB.gs`.

Abstrakt syntax och parsning

För **Num** använder vi den inbyggda typen `Int` i Gofer. För **Var** väljer vi en lista av tecken och definierar så följande typsynonym:

```
type Var = [Char]
```

För var och en av de syntaktiska kategorierna **Aexp**, **Bexp** och **Stm** definierar vi en algebraisk datatyp som täcker de olika produktionerna i While:s syntax.

```
data Aexp = N Int | V Var | Add Aexp Aexp |
           Mult Aexp Aexp | Sub Aexp Aexp

data Bexp = TRUE | FALSE | Eq Aexp Aexp | Le Aexp Aexp |
           Neg Bexp | And Bexp Bexp

data Stm  = Ass Var Aexp | Skip | Comp Stm Stm |
           If Bexp Stm Stm | While Bexp Stm | Repeat Stm Bexp
           Block Stm Var Stm | Raise Var
```

Exempel 1: Fakultetssatsen representeras av följande abstrakta datatyp i Gofer:

```
factorial = Comp (Ass "y" (N 1))
               (While (Neg (Eq (V "x") (N 1)))
                     (Comp (Ass "y" (Mult (V "y") (V "x")))
                           (Ass "x" (Sub (V "x") (N 1)))))
```

Observera att detta är en representation av den abstrakta syntaxen för satsen

```
y := 1; while ~(x = 1) do (y := y * x; x := x - 1)
```

□

Evaluering av uttryck

Vi börjar med att representera värden och tillstånd. De naturliga talen $\mathbf{Z}$ har sin motsvarighet i typen `Int`, så betydelsefunktionen $\mathcal{N}$ blir trivial. Booleska värden i $\mathbf{T}$ representeras av typen `Bool`. Detta leder till följande tpsynonymer:

```
type Z = Int
type T = Bool
```

Mängden **State** definieras som mängden av funktioner från variabler till naturliga tal:

```
type State = Var -> Z
```

Exempel 2: Tillståndet `s_init` avbildar `x` på **3** och alla andra variabler utom `x` på **0**.

```
s_init "x" = 3
s_init y   = 0
```

Observera att vi innesluter ett specifikt variabelnamn med "... " medan `y` kan vara vilken variabel som helst. $\square$

Uppdateringsoperationen $s[x \mapsto v]$ implementeras med följande funktion:

```
update s x v y = v,   if x == y
               = s y, otherwise
```

Betydelsefunktionerna $\mathcal{A}$ och $\mathcal{B}$ döps i implementationen till `a_val` och `b_val`, respektive. De definieras genom direkt översättning av tabell 1.1 och 1.2 i boken:

```
a_val :: Aexp -> State -> Z
b_val :: Bexp -> State -> T

a_val (N n) s      = n
a_val (V x) s      = s x
a_val (Add a1 a2) s = (a_val a1 s) + (a_val a2 s)
a_val (Mult a1 a2) s = (a_val a1 s) * (a_val a2 s)
a_val (Sub a1 a2) s = (a_val a1 s) - (a_val a2 s)

b_val TRUE s       = True
b_val FALSE s      = False
b_val (Eq a1 a2) s  = True,  if a_val a1 s == a_val a2 s
                   = False, if a_val a1 s /= a_val a2 s
b_val (Le a1 a2) s  = True,  if a_val a1 s <= a_val a2 s
                   = False, if a_val a1 s > a_val a2 s
b_val (Neg b) s     = True,  if b_val b s == False
                   = False, if b_val b s == True
b_val (And b1 b2) s = True,  if b_val b1 s == True &&
                           b_val b2 s == True
                   = False, if b_val b1 s == False ||
                           b_val b2 s == False
```

Operationell Semantik i Gofer

I detta avsnitt implementerar vi den naturliga semantiken och den strukturellt operationella semantiken ur kapitel 2. Vi behöver definitionerna från föregående avsnitt men eftersom Gofer inte har något `include`-direktiv måste `appB.gs` antingen laddas in manuellt eller via en projektfil. Förslagsvis skapar ni en projektfil `appC.gp` med följande innehåll:

```
appB.gs
appC.gs
```

För att ladda in filerna kan ni antingen skriva `:p appC.gp` i Gofer eller `gofer + appC.gp` från kommandoraden. Den kod som utvecklas i detta avsnitt placeras i `appC.gs`. Ett labskelett finns att hämta på kursens hemsida.

I kapitel 2 skiljde vi mellan två typer av konfigurationer, nämligen intermediära och slutgiltiga. Detta beskrivs med följande datatyp:

```
data Config = Inter Stm State | Final State
```

Vi börjar med att studera den naturliga semantiken och fortsätter sedan med den strukturella.

Naturlig semantik

Relationen $\rightarrow$ i avsnitt 2.1 i boken beskrivs med en funktion `ns_stm`:

```
ns_stm :: Config -> Config
```

Argumentet till denna funktion motsvarar den vänstra sidan till $\rightarrow$ medan det resultat som produceras motsvarar höger sida. Detta är möjligt eftersom sats 2.9 visade att relationen var deterministisk. Definitionen av `ns_stm` är en rent mekanisk översättning av tabell 2.1 i boken.

```
ns_stm (Inter (Ass x a) s)
  = Final (update s x (a_val a s))
```

```
ns_stm (Inter (Skip) s)
  = Final s
```

```
ns_stm (Inter (Comp ss1 ss2) s)
  = Final s''
```

```

    where Final s'  = ns_stm (Inter ss1 s)
          Final s'' = ns_stm (Inter ss2 s')

ns_stm (Inter (If b ss1 ss2) s)
  = Final s', if b_val b s
  where Final s' = ns_stm (Inter ss1 s)

ns_stm (Inter (If b ss1 ss2) s)
  = Final s', if not(b_val b s)
  where Final s' = ns_stm (Inter ss2 s)

ns_stm (Inter (While b ss) s)
  = Final s'', if b_val b s
  where Final s'  = ns_stm (Inter ss s)
        Final s'' = ns_stm (Inter (While b ss) s')

ns_stm (Inter (While b ss) s)
  = Final s, if not(b_val b s)

```

Observera att i axiomet för tilldelning används `update s x v` för att realisera $s[x \mapsto v]$. Den semantiska funktionen $\mathcal{S}_{\text{ns}}$ kan nu definieras som

```

s_ns ss s = s'
  where Final s' = ns_stm (Inter ss s)

```

Exempel 3: Vi kan exekvera fakultetssatsen från tillståndet `s_init` som avbildade `x` på `3` och alla andra variabler på `0`. Det slutgiltiga tillståndet `s_fac` erhålles på följande vis:

```
s_fac = s_ns factorial s_init
```

För att ta reda på värdet av `y` kan vi evaluera `s_fac "y"`. □

Uppgift 1: Utöka definitionen av `ns_stm` till att innehålla `repeat`-konstruktionen. Redovisa genom att köra följande program

```

y := 1;
repeat (y := y * x;
        x := x - 1)
until (x <= 0)

```

på samma sätt som i föregående exempel. □

Strukturell Operationell Semantik

När vi specificerar den strukturella semantiken måste vi avgöra om $\Rightarrow$ har producerat en intermediär eller slutgiltig konfiguration. För detta ändamål introducerar vi funktionen `is_final`:

```
is_Final (Inter ss s) = False
is_Final (Final s)    = True
```

Relationen $\Rightarrow$ implementeras sedan med funktionen `sos_stm`. Den har följande typ:

```
sos_stm :: Config -> Config
```

Precis som i den naturliga semantiken motsvarar vänster- och högersida till $\Rightarrow$ argument och resultat från funktionen. Återigen ska det betonas att denna implementation endast är möjlig eftersom semantiken är deterministisk (uppgift 2.22 i boken). En översättning av tabell 2.2 leder till följande definition av `sos_stm`:

```
sos_stm (Inter (Ass x a) s)
    = Final (update s x (a_val a s))

sos_stm (Inter (Skip) s) = Final s

sos_stm (Inter (Comp ss1 ss2) s)
    = Inter (Comp ss1' ss2) s', if not(is_Final (sos_stm (Inter ss1 s)))
    where Inter ss1' s' = sos_stm (Inter ss1 s)

sos_stm (Inter (Comp ss1 ss2) s)
    = Inter ss2 s', if is_Final(sos_stm (Inter ss1 s))
    where Final s' = sos_stm (Inter ss1 s)

sos_stm (Inter (If b ss1 ss2) s)
    = Inter ss1 s, if b_val b s

sos_stm (Inter (If b ss1 ss2) s)
    = Inter ss2 s, if not(b_val b s)

sos_stm (Inter (While b ss) s)
    = Inter (If b (Comp ss (While b ss)) Skip) s
```

Funktionen `sos_stm` implementerar ett steg i beräkningen. Den fullständiga härledningssekvensen beräknas av funktionen `deriv_seq` nedan (även om den sekvensen är oändlig!).

```
deriv_seq (Inter ss s)
    = (Inter ss s) : (deriv_seq (sos_stm (Inter ss s)))
deriv_seq (Final s) = [Final s]
```

Den semantiska funktionen $\mathcal{S}_{\text{SOS}}$ definieras nu med Gofer-funktionen `s_sos`:

```
s_sos ss s = s'
    where Final s' = last (deriv_seq (Inter ss s))
```

Exempel 4: Härledningssekvensen för fakultetssatsen i tillståndet `s_init` erhålles med

```
fac_seq = deriv_seq (Inter factorial s_init)
```

Det slutgiltiga tillståndet i härledningssekvensen fås fram med

```
s_fac' = s_sos factorial s_init
```

och värdet av y fås genom att evaluera `s_fac "y"`. □

Uppgift 2: Utöka definitionen av `sos_stm` till att innehålla `repeat`-konstruktionen. Redovisa som i föregående uppgift. □

Denotationssemantik i Gofer

I detta avsnitt implementerar vi den denotationssemantik som beskrivs i bokens kapitel 4. Vi behöver återigen filen `appB.gs` och skapar därför en projektfil `appD.gp` med följande innehåll:

```
appB.gs
appD.gs
```

Hämta labskelettet för `appD.gs` på kursens hemsida.

Direct style semantics

I den implementation som följer kommer vi att använda den inbyggda funktionen `”.”` som representerar funktionskomposition.

De teoretiska grunderna för Gofer bygger till stor del på det material som presenteras i kapitel 4, även om utrymme saknas i detta kompendium för att gå in djupare på detta. En konsekvens är emellertid det faktum att fixpunktsberäkningen kan implementeras på följande enkla vis:

```
fix ff = ff (fix ff)
```

Funktionen $\mathcal{S}_{ds}$ kan nu implementeras med funktionen

```
s_ds :: Stm -> State -> State
```

och de definitioner som vi tagit upp på föreläsningen (och som i något omskriven form återfinns i bokens tabell 4.1) kan översättas nästan mekaniskt till

```
s_ds (Ass x a)      = \s -> update s x (a_val a s)
s_ds Skip           = \s -> s
s_ds (Comp ss1 ss2) = (s_ds ss2) . (s_ds ss1)
s_ds (If b ss1 ss2) = \s -> if b_val b s then s_ds ss1 s else s_ds ss2 s
s_ds (While b ss)   = fix (\g s -> if b_val b s then g (s_ds ss s) else s)
```

Exempel 5: Vi kan återigen illustrera semantiken genom att ”köra” fakultetssatsen i det initiala tillståndet `s_init`:

```
s_final = s_ds factorial s_init
```

□

Uppgift 3: Utöka `s_ds` till att klara av `repeat`-konstruktionen. Redovisa som förut. □

Fortsättningssemantik

För att representera en fortsättning introducerar vi följande typsynonym:

```
type Cont = State -> State
```

En fortsättningsomgivning (boken, s. 130) är en avbildning från identifierare till en fortsättning och leder till följande typsynonym:

```
type Env_E = Var -> Cont
```

Produktionerna

$$S \rightarrow \dots \mid \text{begin } S_1 \text{ handle } e : S_2 \text{ end} \mid \text{raise } e$$

har sin motsvarighet i den abstrakta syntaxen, datatypen `Stm` i `appB.gs`

```
data Stm = Ass Var Aexp | Skip | Comp Stm Stm |  
          If Bexp Stm Stm | While Bexp Stm | Repeat Stm Bexp  
          Block Stm Var Stm | Raise Var
```

där komponenten `Var` i `Block` och `Raise` representerar identifieraren e i produktionerna.

Uppgift 4: Skriv en funktion `s_cs` för att realisera fortsättningssemantiken $\mathcal{S}_{cs}$ i boken. Testa er kod med programmet

```
begin  
  if x < 0 then skip else raise exit  
  handle exit: x := 0  
end
```

genom att köra följande:

```
test = Block (If (Le (V "x") (N 0)) Skip (Raise "exit"))  
        "exit" (Ass "x" (N 0))  
  
e_init x = id -- initial exception environment  
  
c_init = id   -- initial continuation  
  
s_final' = s_cs test e_init c_init s_init
```

Vad har `x` för värde i `s_final'`?

□